To Add spring Rest Data, in pom.xml file:

```
package com.jloka.jloka.repositories;

import java.util.List;

import org.springframework.data.repository.CrudRepository;
import org.springframework.data.repository.PagingAndSortingRepository;
import org.springframework.data.rest.core.annotation.RepositoryRestResource;

import com.jloka.jloka.models.Customer;

@RepositoryRestResource(collectionResourceRel = "customers", path = "customers")
public interface RestCustomerRepository extends
PagingAndSortingRepository<Customer, Long>,
    CrudRepository<Customer, Long> {
    List<Customer> findByLastName(String name);
}
```

****

The models for all relationship:

The model for User:

```
package com.jloka.jloka.models;

import jakarta.persistence.*;
import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(nullable = false, unique = true)
    private String username;
    @Column(nullable = false)
    private String email;
    // One-to-One relationship with UserProfile
```

```
    @OneToOne(mappedBy = "user", cascade = CascadeType.ALL, fetch =
FetchType.LAZY)
    private UserProfile profile;
    // One-to-Many relationship with Post
    @OneToMany(mappedBy = "author", cascade = CascadeType.ALL, fetch =
FetchType.LAZY)
    private List<Post> posts = new ArrayList<>();
    // Many-to-Many relationship with Role
    @ManyToMany(fetch = FetchType.LAZY)
    @JoinTable(
        name = "user_roles",
        joinColumns = @JoinColumn(name = "user_id"),
        inverseJoinColumns = @JoinColumn(name = "role_id")
    )
    private Set<Role> roles = new HashSet<>();
    // Constructors
    public User() {}

    public User(String username, String email) {
        this.username = username;
        this.email = email;
    }

    // Getters and Setters
    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getUsername() {
        return username;
    }

    public void setUsername(String username) {
        this.username = username;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
```

```
        this.email = email;
    }

    public UserProfile getProfile() {
        return profile;
    }

    public void setProfile(UserProfile profile) {
        this.profile = profile;
        profile.setUser(this);
    }

    public List<Post> getPosts() {
        return posts;
    }

    public void setPosts(List<Post> posts) {
        this.posts = posts;
    }

    public void addPost(Post post) {
        posts.add(post);
        post.setAuthor(this);
    }

    public Set<Role> getRoles() {
        return roles;
    }

    public void setRoles(Set<Role> roles) {
        this.roles = roles;
    }

    public void addRole(Role role) {
        roles.add(role);
        role.getUsers().add(this);
    }

    public void removeRole(Role role) {
        roles.remove(role);
        role.getUsers().remove(this);
    }
}
```

The profile model is:

```
package com.jloka.jloka.models;

import java.time.LocalDate;
import jakarta.persistence.*;

@Entity
@Table(name = "user_profiles")
public class UserProfile {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "first_name")
    private String firstName;

    @Column(name = "last_name")
    private String lastName;

    @Column(name = "date_of_birth")
    private LocalDate dateOfBirth;

    @Column(name = "address")
    private String address;

    @Column(name = "phone_number")
    private String phoneNumber;

    // One-to-One relationship with User
    @OneToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id", nullable = false, unique = true)
    private User user;

    // Constructors
    public UserProfile() {}

    public UserProfile(String firstName, String lastName, LocalDate dateOfBirth)
{
        this.firstName = firstName;
        this.lastName = lastName;
        this.dateOfBirth = dateOfBirth;
    }

    // Getters and Setters
```

```java
public Long getId() {
    return id;
}

public void setId(Long id) {
    this.id = id;
}

public String getFirstName() {
    return firstName;
}

public void setFirstName(String firstName) {
    this.firstName = firstName;
}

public String getLastName() {
    return lastName;
}

public void setLastName(String lastName) {
    this.lastName = lastName;
}

public LocalDate getDateOfBirth() {
    return dateOfBirth;
}

public void setDateOfBirth(LocalDate dateOfBirth) {
    this.dateOfBirth = dateOfBirth;
}

public String getAddress() {
    return address;
}

public void setAddress(String address) {
    this.address = address;
}

public String getPhoneNumber() {
    return phoneNumber;
}

public void setPhoneNumber(String phoneNumber) {
```

```
        this.phoneNumber = phoneNumber;
    }

    public User getUser() {
        return user;
    }

    public void setUser(User user) {
        this.user = user;
    }
}
```

The role model is:

```
package com.jloka.jloka.models;

import jakarta.persistence.*;
import java.util.HashSet;
import java.util.Set;

@Entity
@Table(name = "roles")
public class Role {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String name;

    @Column(columnDefinition = "TEXT")
    private String description;

    // Many-to-Many relationship with User
    @ManyToMany(mappedBy = "roles", fetch = FetchType.LAZY)
    private Set<User> users = new HashSet<>();

    // Constructors
    public Role() {}

    public Role(String name, String description) {
        this.name = name;
        this.description = description;
```

```
    }

    // Getters and Setters
    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getDescription() {
        return description;
    }

    public void setDescription(String description) {
        this.description = description;
    }

    public Set<User> getUsers() {
        return users;
    }

    public void setUsers(Set<User> users) {
        this.users = users;
    }
}
```

The tag model is:

```
package com.jloka.jloka.models;

import jakarta.persistence.*;
import java.util.HashSet;
import java.util.Set;
```

```java
@Entity
@Table(name = "tags")
public class Tag {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String name;

    // Many-to-Many relationship with Post
    @ManyToMany(mappedBy = "tags", fetch = FetchType.LAZY)
    private Set<Post> posts = new HashSet<>();

    // Constructors
    public Tag() {}

    public Tag(String name) {
        this.name = name;
    }

    // Getters and Setters
    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public Set<Post> getPosts() {
        return posts;
    }

    public void setPosts(Set<Post> posts) {
        this.posts = posts;
```

```
    }
}
```

The post model is:

```
package com.jloka.jloka.models;

import jakarta.persistence.*;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

@Entity
@Table(name = "posts")
public class Post {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String title;

    @Column(columnDefinition = "TEXT", nullable = false)
    private String content;

    @Column(name = "created_at")
    private LocalDateTime createdAt;

    // Many-to-One relationship with User
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "author_id", nullable = false)
    private User author;

    // One-to-Many relationship with Comment
    @OneToMany(mappedBy = "post", cascade = CascadeType.ALL, fetch =
FetchType.LAZY)
    private List<Comment> comments = new ArrayList<>();

    // Many-to-Many relationship with Tag
    @ManyToMany(fetch = FetchType.LAZY)
    @JoinTable(
```

```
    name = "post_tags",
    joinColumns = @JoinColumn(name = "post_id"),
    inverseJoinColumns = @JoinColumn(name = "tag_id")
)
private Set<Tag> tags = new HashSet<>();

// Constructors
public Post() {}

public Post(String title, String content, User author) {
    this.title = title;
    this.content = content;
    this.author = author;
    this.createdAt = LocalDateTime.now();
}

@PrePersist
protected void onCreate() {
    createdAt = LocalDateTime.now();
}

// Getters and Setters
public Long getId() {
    return id;
}

public void setId(Long id) {
    this.id = id;
}

public String getTitle() {
    return title;
}

public void setTitle(String title) {
    this.title = title;
}

public String getContent() {
    return content;
}

public void setContent(String content) {
    this.content = content;
}
```

```java
public LocalDateTime getCreatedAt() {
    return createdAt;
}

public void setCreatedAt(LocalDateTime createdAt) {
    this.createdAt = createdAt;
}

public User getAuthor() {
    return author;
}

public void setAuthor(User author) {
    this.author = author;
}

public List<Comment> getComments() {
    return comments;
}

public void setComments(List<Comment> comments) {
    this.comments = comments;
}

public void addComment(Comment comment) {
    comments.add(comment);
    comment.setPost(this);
}

public Set<Tag> getTags() {
    return tags;
}

public void setTags(Set<Tag> tags) {
    this.tags = tags;
}

public void addTag(Tag tag) {
    tags.add(tag);
    tag.getPosts().add(this);
}

public void removeTag(Tag tag) {
    tags.remove(tag);
```

```
        tag.getPosts().remove(this);
    }
}
```

The comment model:

```
package com.jloka.jloka.models;

import jakarta.persistence.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "comments")
public class Comment {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(columnDefinition = "TEXT", nullable = false)
    private String content;

    @Column(name = "created_at")
    private LocalDateTime createdAt;

    // Many-to-One relationship with Post
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "post_id", nullable = false)
    private Post post;

    // Many-to-One relationship with User
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id", nullable = false)
    private User user;

    // Constructors
    public Comment() {}

    public Comment(String content, Post post, User user) {
        this.content = content;
        this.post = post;
        this.user = user;
        this.createdAt = LocalDateTime.now();
    }
```

```java
@PrePersist
protected void onCreate() {
    createdAt = LocalDateTime.now();
}

// Getters and Setters
public Long getId() {
    return id;
}

public void setId(Long id) {
    this.id = id;
}

public String getContent() {
    return content;
}

public void setContent(String content) {
    this.content = content;
}

public LocalDateTime getCreatedAt() {
    return createdAt;
}

public void setCreatedAt(LocalDateTime createdAt) {
    this.createdAt = createdAt;
}

public Post getPost() {
    return post;
}

public void setPost(Post post) {
    this.post = post;
}

public User getUser() {
    return user;
}

public void setUser(User user) {
    this.user = user;
```

```
    }
}
```